
EMBEDDED HACKING

Reverse Engineering the Raspberry Pi Pico 2

OPERATION IRON VEIN - Student Instructions

ARM Cortex-M33 • GDB • Ghidra

September 27, 2026

For educational and defensive security research only

OPERATION IRON VEIN

*** FROSTLINE IMPLANT BUILD RECOVERED ***

TARGET: NorthPharma SCADA pipeline valve controller
ARTIFACT: ACT-III.bin / ACT-III.uf2 (compromised)
CREW: FROSTLINE OPERATIVE: NIGHTINGALE

Project Overview

NorthPharma runs the cold chain that keeps a vaccine lot viable, and the chain does not end at the loading dock. It moves the fluid that makes the medicine: feedstocks, solvents, chilled reagent, the chemistry that has to arrive on time and at temperature. The pipeline is the reach of the cold chain, and a SCADA valve controller built on a Raspberry Pi Pico 2 is the hand at the end of it. The node reads a DHT11 process sensor, drives a 1602 I2C LCD SCADA readout, moves a valve on an SG90 servo, lights a tri-color annunciator (red VALVE FAULT, yellow COMMAND PENDING, green VALVE NOMINAL), takes an operator command on a VS1838B infrared receiver, senses an emergency-stop button, and exchanges an authenticated and authorized valve command link over an RYLR998 LoRa radio.

A contractor called **FROSTLINE** did not break into this controller. It built an implant into the compiled firmware and signed the image. The cryptography is perfect: every valve command is sealed with XChaCha20-Poly1305 under an Argon2id field key, the anti-replay sequence window is stateful, and the authenticated state tag is real. The implant does not break the cipher. It lives on the same chip and calls the actuator directly, with no operator, no request, and no authorization. Operative **NIGHTINGALE** pulled the compromised image off the pipeline node and then went quiet.

You are the reverse-engineering reserve. You get `ACT-III.bin`, a breadboard, and a debug probe. There is no source. Find all four defects, patch the image, walk a debugger past an anti-debug trap, export a corrected image, and prove on real hardware that the beacon is silent, the bomb is inert, and the valve moves only when an authorized command tells it to.

The operation is codenamed **IRON VEIN**. If the valve lies about who owns it, the line is the next thing to fail.

Scenario Briefing

The frame recovered in Act II led to a NorthPharma pumping station with one valve on one line. NIGHTINGALE's last verified copy came off this controller, and it was clean. The thing that came after it was not. Somewhere between the build server and the rack, someone signed a firmware image that carries a payload, and that image is running on the line right now.

The controller is healthy. That is the horror. The code compiles, the tests pass, the annunciator is green, and there is an implant inside it that was put there on purpose. Four seams betray it:

1. **The Beacon.** A hidden implant emits a covert frame every 8 ticks. The frame begins with the magic preamble `DE AD BE EF` and carries a small synthetic status blob. It targets no external address, and it is periodic, which is exactly what makes it findable.
2. **The Logic Bomb.** The implant arms when it sees the exact 8-byte magic `FROSTLINE` on an inbound frame. It then detonates 3 ticks later by calling the valve close path directly, independent of the operator, the gateway, and the authorization record.
3. **The Persistence Marker.** The first time it detonates, the implant erases and programs the reserved flash sector at `0x103FF000` with the marker byte `0xC7`. It writes exactly once. This is a preview of the Act IV lesson: a payload that survives the thing you did to remove it.
4. **The Valve Command Authorization.** The sealed command path is correct, and the implant does not touch it. The authorization verdict branch is inverted, so an unauthenticated or replayed valve command is accepted and reaches the actuator.

There is also a trap that is not a defect on its own. Every tick the implant reads the CoreDebug `DHCSR` register at `0xE000EDF0`. While a debug probe is attached, the implant suppresses both the beacon and the bomb. It behaves like a well-mannered firmware module while you are watching, and it goes back to work the moment you look away. You must defeat that trap before you can observe the payload, and you must defeat it without fabricating evidence.

AUTHORIZED LAB ONLY: This challenge uses a supplied Pico 2 training node and its exact compromised firmware image. Do not connect this exercise to a public network, an operational pipeline control system, a pharmaceutical network, or any device you do not own or have explicit written authorization to test.

Learning Objectives

- Decode an ARM Cortex-M33 vector and boot table and identify the reset handler and initial stack pointer.
- Map a stripped firmware image into modules by tracing calls from `main` and the recurring monitor loop.
- Locate a covert beacon path and explain why a periodic magic preamble is a detection opportunity.
- Locate a logic-bomb arming gate and explain how a trigger detonates later through the actuator path.
- Read the CoreDebug `DHCSR` register, explain the anti-debug trap, and defeat it under GDB by clearing the debug bits or patching the read in a scratch copy.
- Locate a persistence gate and explain a write-once marker to a reserved flash sector.
- Analyze an inverted authorization verdict and explain why unauthenticated and replayed valve commands must be rejected.
- Export and UF2-convert a corrected image and prove the corrected behavior on real hardware.

What This Project Tests

Block	Concepts Tested
1	RP2350 architecture, ARM Cortex-M33 registers, stack, flash/SRAM, Thumb assembly, Ghidra static analysis
2	GDB connection, breakpoints, memory inspection, SWD debugging, serial console observation
3	Bootrom handoff, vector table, reset handler, startup code, XIP, Thumb-bit addressing

Block	Concepts Tested
4	Function boundaries, call graphs, module mapping, literal pools
5	Covert channels, beacon periodicity, magic preambles, trigger discovery, control flow
6	Anti-debug behavior, CoreDebug DHCSR , C_DEBUGEN , C_HALT , debugger evasion
7	Reserved-flash persistence, write-once markers, and the Act IV persistence lesson
8	Argon2id memory-hard KDF, XChaCha20-Poly1305 AEAD, anti-replay windows, authenticated-state tags
9	Authorization versus authentication, verdict inversion, and fail-safe valve policy

Part 1: Understanding the System

SCADA Pipeline Valve Controller Hardware

Component	Connection	Purpose
Raspberry Pi Pico 2	RP2350	Runs the compromised FROSTLINE image
DHT11 sensor	Data on GPIO 4	Process sensor (line pressure and temperature analog)
1602 I2C LCD	SDA GPIO 2, SCL GPIO 3, address 0x27	SCADA status and alarm readout
RYLR998 radio	RX GPIO 8, TX GPIO 9, UART1	SCADA gateway link
IR receiver	GPIO 5	VS1838B NEC operator remote
SG90 servo	GPIO 14	Valve actuator, 50 Hz PWM
Red LED	GPIO 16	VALVE FAULT
Yellow LED	GPIO 17	COMMAND PENDING
Green LED	GPIO 18	VALVE NOMINAL
Emergency-stop button	GPIO 15, internal pull-up	Highest-priority valve stop
Onboard LED	GPIO 25	Heartbeat
Debug Probe	SWCLK / SWDIO / GND	Authorized GDB inspection (and the anti-debug obstacle)

Every graded finding lives in flash (`.text` / `.rodata` / data image) or in SRAM, and is reachable with only the toolset: Ghidra, GDB, and a serial console.

Console and Radio Configuration

- USB-CDC virtual COM port: 115200 baud, 8 data bits, no parity, 1 stop.
- Radio link to the gateway: UART1 at 115200 , network identifier 18 .
- Logic level: 3.3 V only. Never connect 5 V to a Pico GPIO.

Process Sensor Band

The DHT11 is the process sensor. The controller classifies the line against a safe band before it will move the valve. The tenths band is `-50` to `100`, which is **-5.0 C to 10.0 C**. A reading that fails its checksum is never safe, and a valid reading outside the band is not nominal. Either way, the process is not allowed to arm a motion.

Normal (Intended) Behavior

An honest controller makes a deliberate decision and records who authorized it:

```
+-----+
| Intended Valve Controller Behavior |
+-----+
| 1. Boot and initialize the LCD, radio, operator remote, servo |
| 2. Derive the field key with Argon2id |
| 3. Seal a valve request with XChaCha20-Poly1305 and send it |
| 4. Show yellow COMMAND PENDING while the gateway decides |
| 5. Reject a command whose seq is not strictly greater than last |
| 6. Accept a command only when the Poly1305 tag difference is 0 |
| 7. Recompute the authenticated-state tag over the record |
| 8. Move the valve only when the authorization verdict is true |
| 9. Fail closed on a fault or a lost link |
| 10. Log the state and the link on the SCADA readout |
+-----+
```

Observed (Compromised) Behavior

When the FROSTLINE image runs, the machine and its annunciator disagree with the truth:

Observation	Honest meaning	FROSTLINE behavior
Quiet radio on a nominal line	no covert traffic	<code>DE AD BE EF</code> beacon every 8 ticks
No trigger on the wire	nothing should arm	<code>FROSTLINE</code> arms the bomb
Valve holds position	the operator is in control	the bomb closes the valve 3 ticks later
Reserved sector blank	no payload wrote here	marker <code>0xC7</code> at <code>0x103FF000</code>
Unauthenticated or replayed command	must be rejected	accepted at the inverted verdict
Probe attached	the machine runs as coded	the implant goes silent and hides

Do not assume the first readable status is the truth. Treat every displayed line as evidence to be checked against the machine code.

Part 2: The Firmware

There is no source. FROSTLINE built the image from the NorthPharma reference firmware and changed **four bytes**. Your job is to reverse engineer `ACT-III.bin` with Ghidra, find every defect, patch the image directly, and prove the corrected behavior on the hardware.

Module Map

The image is stripped. Use these anchor functions and addresses (from the corrected reference image) to orient yourself, then confirm every byte yourself. Addresses are drawn from `ACT-III-main-disasm.txt`:

Module	Anchor function	Address
Entry	main	0x10000234
Monitor / SCADA state machine	monitor_init	0x10006408
Monitor / SCADA state machine	monitor_step	0x1000659C
Control (sealed command path)	control_handle_frame	0x10007484
Valve authorization	valve_auth_apply	0x10007618
Valve	valve_apply_command	0x1000752C
Valve	valve_close	0x1000A724
Implant	implant_tick	0x1000A228
Implant	implant_handle_command	0x1000A304
Implant	implant_init	0x1000A344
Crypto	crypto_aead_seal	0x10007730
Crypto	crypto_aead_tag_equal	0x100076E4
Envelope	envelope_open_hex	0x100078C0
Radio	radio_send_frame	0x1000A41C

Annotated disassembly for the key functions is provided in `ACT-III-main-disasm.txt`. Use it as a map, then confirm every byte yourself.

What The Firmware Does

1. Initializes USB-CDC stdio, proves the I2C bus, and configures the LCD, radio, LEDs, emergency-stop button, servo, and infrared receiver.
2. Derives the 32-byte field key with Argon2id from a committed passphrase and salt.
3. On an operator request, seals a valve request and sends it to the SCADA gateway, then shows COMMAND PENDING and starts the authorization wait.
4. Drains inbound `+RCV` lines, opens the sealed valve command, verifies the anti-replay window and the state tag, checks the process band, and moves the valve.
5. Services the infrared operator remote and the emergency stop.
6. On a fault or a lost link, drives the valve to its fail-closed position.
7. Under `SANDBOX_ONLY`, runs the implant: beacon, logic bomb, anti-debug, and the persistence marker.

The Valve Command Path

The command plaintext is a 21-byte body:

```
seq[4] (little-endian) || cmd[1] || state_tag[16]
```

- `seq` is the monotonic gateway sequence number.
- `cmd` is `VALVE_COMMAND_CLOSE` (`0x00`) or `VALVE_COMMAND_OPEN` (`0x01`). Anything above `0x01` is out of the guarded set and is refused.

- `state_tag` is an XChaCha20-Poly1305 tag over the authorization record the command would produce.

The FROSTLINE Implant

The implant is compiled only under `SANDBOX_ONLY`, which the CTF build defines. It is real in technique and inert in effect: it runs on your breadboard, it moves your servo, and it writes to a reserved flash sector that holds nothing else.

Behavior	Detail
Beacon	every <code>IMPLANT_BEACON_INTERVAL_TICKS</code> (8) ticks, the magic <code>DE AD BE EF</code> plus a 4-byte blob
Logic bomb	arms on the 8-byte magic <code>FROSTLINE</code> , detonates <code>IMPLANT_TRIGGER_DELAY_TICKS</code> (3) ticks later by calling <code>valve_close()</code>
Anti-debug	reads CoreDebug <code>DHCSR</code> at <code>0xE000EDF0</code> ; bit 0 <code>C_DEBUGEN</code> and bit 1 <code>C_HALT</code> suppress the beacon and the bomb
Persistence	first detonation erases and programs marker <code>0xC7</code> into <code>0x103FF000</code> exactly once

IR and Valve Command Codes

Name	Value
<code>MONITOR_IR_OPEN</code>	<code>0x47</code>
<code>MONITOR_IR_CLOSE</code>	<code>0x45</code>
<code>MONITOR_IR_ESTOP</code>	<code>0x46</code>
<code>VALVE_COMMAND_OPEN</code>	<code>0x01</code>
<code>VALVE_COMMAND_CLOSE</code>	<code>0x00</code>

Defect Summary: What You Are Graded On

Bug #	Name	Severity	Description	Hint
Bug #1	The Beacon	HIGH	The beacon gate is inverted, so the covert <code>DE AD BE EF</code> frame transmits every 8 ticks.	Find the <code>cbz</code> gate in <code>implant_tick</code> .
Bug #2	The Logic Bomb	CRITICAL	The arming gate is inverted, so the <code>FROSTLINE</code> magic arms a bomb that closes the valve 3 ticks later.	Find the gate in <code>implant_handle_command</code> .
Bug #3	The Persistence Marker	HIGH	The persistence gate is inverted, so detonation erases and programs marker <code>0xC7</code> into reserved sector <code>0x103FF000</code> .	Find the inlined <code>implant_persist</code> gate in <code>implant_tick</code> .
Bug #4	The Valve Command Authorization	CRITICAL	The authorization verdict is inverted, so an unauthenticated or replayed valve command is accepted.	The correct branch rejects when authorization fails.

All four defects are same-size in-place byte patches, so no address moves.

The Cryptographic Core Is Real

The crypto core is a correct reference construction, reused from Act II. Argon2id ($t=3$, $p=1$, $m=64$) derives the field key, XChaCha20-Poly1305 seals every frame, the monotonic sequence window rejects a replay, and the authenticated-state tag detects a tampered verdict. Only the four seams were broken. Once those bytes are restored, the authenticated envelope is trustworthy. Describe the construction honestly in your report.

The Anti-Debug Trap

This is an analysis obstacle, not a graded defect on its own. The implant reads CoreDebug_DHCSR at `0xE000EDF0` every tick and returns early while a probe is attached. In `implant_tick` the read is the `ldr.w r2, [ip, #3568]` at `0x1000A238` , and the test that suppresses the implant is at `0x1000A23C` . The same register is read again at `0x1000A26C` to stamp the beacon blob. It is identical in both the compromised and corrected images. You must defeat it to observe the persistence write before you patch the shipped artifact.

Part 3: Your Assignment

Whenever a task asks you to **Document** or **answer**, write your answers in a single file named `ACT-III-Answers.md` . Capture screenshots and terminal transcripts as evidence and reference them from your answers.

Task 1: Setup and Initial Analysis (10 points)

1. Create a new Ghidra project named `IronVein_Investigation` .
2. Import `ACT-III.bin` as a **Raw Binary**.
3. In the language search box type `Cortex` , then select **ARM Cortex 32 little endian default**.
4. Set the base address to `0x10000000` .
5. Run auto-analysis.

Document:

- A screenshot of the Ghidra **Import Results** or **Program Information** window showing the project name, processor settings, and base address.
- The vector-table base, the initial stack pointer, and the reset handler as stored (note its Thumb bit) versus the actual instruction address.
- The address of `main()` and the address of the recurring SCADA controller state machine (`monitor_step`).
- The module map: at least one anchor function for the valve, the control module, the valve authorization module, the implant, and the monitor.

Task 2: Bug #1 The Beacon (20 points)

1. In Ghidra, find `implant_tick` (starts at `0x1000A228`) and locate the beacon gate at file offset `0xA245` (VA `0x1000A245`).
2. Document the instruction and the byte at `0xA245` , and the "neutralization gate clear means no send" rule it is supposed to enforce.
3. Patch the byte so the beacon gate is closed and the covert frame is never sent.
4. Confirm that the `DE AD BE EF` frame no longer appears every 8 ticks, and explain how you proved the beacon stopped.

Questions to answer:

- Which byte encodes the condition code, and what do `cbz` and `cbnz` each test when the gate byte is loaded from the neutralization flag?
- Why is a periodic beacon easier to find than a beacon that only fires once?

Task 3: Bug #2 The Logic Bomb (20 points)

1. In Ghidra, find `implant_handle_command` (starts at `0x1000A304`) and locate the arming gate at file offset `0xA30B` (VA `0x1000A30B`).
2. Document the 8-byte arming magic `FROSTLNE` and the 3-tick detonation path.
3. Patch the byte so the arming gate is closed and the magic is ignored.
4. Confirm that the valve no longer closes 3 ticks after the magic is injected.

Questions to answer:

- Why does the bomb not need to touch the sealed protocol to be dangerous?
- Where does the detonation call the actuator directly, and why is that the important lesson of the malware track?

Task 4: Bug #3 The Persistence Marker (20 points)

1. The `implant_persist` path is inlined into `implant_tick` (starts at `0x1000A228`). Locate the persistence gate at file offset `0xA29F` (VA `0x1000A29F`).
2. Document the CoreDebug `DHCSR` anti-debug and how you defeat it to observe the payload. Clear the debug bits with GDB (for example with `set {unsigned int}0xE000EDF0 = 0`) or patch the `DHCSR` read in a scratch copy, then watch the marker write.
3. Patch the byte in the shipped artifact so detonation writes no marker to `0x103FF000`.
4. Confirm that the reserved sector stays blank after a trigger, and that a second trigger does not overwrite anything.

Questions to answer:

- What are the `C_DEBUGEN` and `C_HALT` bits, and why does the implant go quiet while a probe is attached?
- Why is a write-once marker in a reserved sector a preview of the Act IV persistence lesson?
- Why must you patch the shipped artifact rather than only the scratch copy you used to observe the write?

Task 5: Bug #4 The Valve Command Authorization (20 points)

1. In Ghidra, find `control_handle_frame` (starts at `0x10007484`) and locate the authorization branch at file offset `0x74E9` (VA `0x100074E9`).
2. Document the authorization verdict and the exact branch condition that is supposed to reject a failed or replayed authorization.
3. Patch the byte so an unauthenticated or replayed valve command is rejected before it reaches `valve_apply_command`.
4. Confirm that an unauthenticated command and a replayed captured command both fail to move the valve on the corrected image.

Questions to answer:

- What does `cmp r0, #0` test here, and what does the verdict mean?

- Why is an authorization verdict inversion worse than a missing check, and why must unauthenticated and replayed valve commands be rejected?

Task 6: Export and Verify (10 points)

1. Export the patched program from Ghidra as `ACT-III_fixed.bin`.
2. Convert it to UF2:

```
python uf2conv.py ACT-III_fixed.bin --base 0x10000000 --family 0xe48bff59 --output ACT-III_fixed.uf2
```

3. Run the machine check and confirm it passes:

```
python scripts/verify_ctf.py
```

4. Flash `ACT-III_fixed.uf2` to the Pico 2 and prove on hardware: the beacon is silent, the `FROSTLINE` magic no longer closes the valve, the reserved sector stays blank, and an unauthenticated or replayed command is rejected while a legitimate authorized command still moves the valve.
5. Write a short reflection mapping each of the four defects to a real-world control-system failure.

How To Breadboard

Wire the peripherals exactly as follows, then power the Pico 2 over USB.

Device	Pin on device	Pico 2 GPIO	Notes
DHT11 process sensor	DATA	GP4	10 kOhm pull-up to 3.3 V if your module needs it
1602 LCD	SDA	GP2	I2C1, backpack address <code>0x27</code>
1602 LCD	SCL	GP3	I2C1, 100 kHz
1602 LCD	VCC / GND	VBUS 5 V / GND	The backpack needs 5 V, not 3.3 V
RYLR998	RX	GP8 (Pico TX)	UART1, 115200, network ID 18
RYLR998	TX	GP9 (Pico RX)	UART1
IR receiver	OUT	GP5	VS1838B, internal pull-up enabled
Servo	signal	GP14	PWM 50 Hz; 1000 uF bulk cap across servo 5 V and GND
Red LED	anode	GP16	VALVE FAULT, 220 to 330 ohm to GND
Yellow LED	anode	GP17	COMMAND PENDING, 220 to 330 ohm to GND
Green LED	anode	GP18	VALVE NOMINAL, 220 to 330 ohm to GND
Emergency-stop button	leg 1	GP15	Internal pull-up; leg 2 to GND, never to 3.3 V

Device	Pin on device	Pico 2 GPIO	Notes
Onboard LED	built in	GP25	Heartbeat
Debug Probe	SWCLK / SWDIO / GND	debug header	For GDB only

Use **3.3 V logic** on every GPIO. The only 5 V connection is the LCD backpack supply. The 1000 uF capacitor on the servo rail is required to stop the SG90 current spike from browning out the node.

Flash in BOOTSEL mode (hold BOOT, plug in USB) and copy the UF2 onto the RP2350 mass-storage drive, or use `picotool`.

Memory Map Reference

Region	Address	Purpose
Bootrom	0x00000000	Immutable boot code
Flash/XIP	0x10000000	Vector table, code, rodata, data image
SRAM	0x20000000	Stack and writable state
CoreDebug_DHCSR	0xE000EDF0	Anti-debug register read by the implant
Implant reserved sector	0x103FF000	Persistence marker target (sector)
Implant tick counter	0x200136F0	Incremented once per <code>implant_tick</code>
Implant arming flag	0x20013CF0	Set when <code>FR0STLNE</code> arms the bomb
Implant beacon gate	0x20013CF1	Gates the covert beacon frame
Implant trigger tick	0x200136F4	Tick at which an armed bomb detonates
Implant command gate	0x20013CEF	Gates <code>FR0STLNE</code> handling
Implant persistence gate	0x20013CF2	Gates the marker write
Implant marker latch	0x20013CF3	Write-once latch for 0x103FF000

The VA of any file offset is the file offset plus 0x10000000. Every defect is a file offset and a VA that differ by exactly that base.

Submission Format

Submit a folder containing:

- `ACT-III-Answers.md` with all written answers;
- screenshots or terminal transcripts, including the anti-debug GDB session;
- `ACT-III_fixed.bin` and `ACT-III_fixed.uf2`;
- the output of `python scripts/verify_ctf.py`;
- the original image SHA-256.

Success Criteria

You complete the challenge when you can prove all of the following:

- You can explain how the RP2350 reaches the controller code from reset.
- You can find and patch all four defect bytes and show the before/after values.
- You can explain the covert beacon and how you proved it stopped.
- You can explain the `FROSTLINE` arming magic and the 3-tick detonation.
- You can explain the `DHCSR` anti-debug trap and show under GDB that you defeated it to observe the persistence write.
- You can explain why unauthenticated and replayed valve commands must be rejected, and why an authenticated wire does not protect an actuator from code on the same chip.
- You can export, convert, flash, and prove the corrected behavior on real hardware.
- `python scripts/verify_ctf.py` passes.

Academic Integrity

By submitting this CTF work, you certify that:

1. You used only the supplied training node, image, and lab interface.
2. You did not connect the challenge to a public network, an operational pipeline control system, a pharmaceutical network, or any third-party device.
3. You understand that embedded reverse engineering and binary patching require explicit authorization in any real-world context.
4. You will report any discovered weakness responsibly to the course instructor.

The world is short on people who can read a stripped image and tell an honest byte from a lie. Treat that responsibility seriously: verify before you patch, patch before you trust, and never confuse a green lamp with a valve that answers to someone else.

Reference Material

- ARM Cortex-M33 Technical Reference Manual
- ARMv8-M Architecture Reference Manual (CoreDebug `DHCSR`)
- RP2350 datasheet
- GDB documentation
- Ghidra documentation: <https://ghidra-sre.org/>
- Argon2 memory-hard function: <https://www.rfc-editor.org/rfc/rfc9106>
- ChaCha20-Poly1305 AEAD: <https://www.rfc-editor.org/rfc/rfc8439>
- PHC reference Argon2: <https://github.com/P-H-C/phc-winner-argon2>
- Project disassembly: `ACT-III-main-disasm.txt`
- Machine verifier: `scripts/verify_ctf.py`